

LAB3 FPGA3

실험 후 보고서

실험자 : 권진혁

학번 : 2023440007

학과 : 전자전기컴퓨터공학부

실험 일시 : 2026. 09. 28.

Duty Level은 총 10단계이며

시뮬레이션 파형에서 Testbench는 다음 조건을 검증하였다.

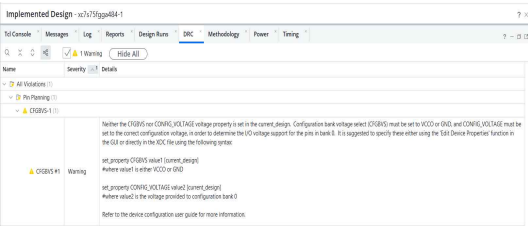
- R 채널 Duty = 20%
- G 채널 Duty = 50%
- B 채널 Duty = 80%
- R 버튼을 1회 추가 입력하여 20% → 30% 변화 확인

교안의 Testbench에서 검증하는 기댓값은 ‘measure(2,5,8), measure(3,5,8)’이며 이며 PWM 주기를 10단계(Level=10)로 나누었을 때 각각 High 구간 개수가 2, 5, 8이 되는 것을 확인하였다.

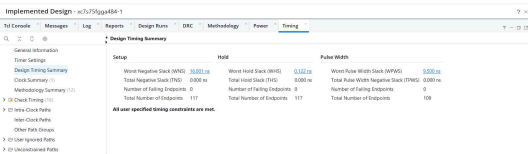
파형에서는

- led_r : High 비율 약 20%
- led_g : High 비율 약 50%
- led_b : High 비율 약 80% 로 관측되었으며, 추가로 R 버튼 입력 후
- R : 20% → 30%
- G : 50% 유지
- B : 80% 유지되는 것을 확인하였다.

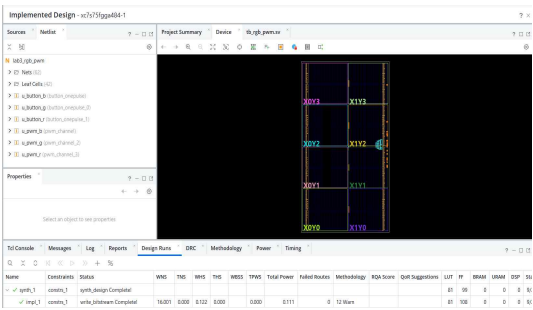
· DRC



· Timing



· Bitstream



· 실제 보드 관찰

보드에 bitstream을 다운로드한 후 RGB 버튼을 눌러 확인하였다.

- R 버튼 : Red 밝기만 증가
 - G 버튼 : Green 밝기만 증가
 - B 버튼 : Blue 밝기만 증가하였다.
- 세 채널은 서로 독립적으로 동작하였으며 RGB 혼합색도 정상적으로 출력되었다.

· 계산값

PWM은 10단계로 구성되어 있으므로 Duty Cycle은 다음과 같다.

Level	Duty Cycle
2	20%
5	50%
8	80%

· 실측값과 차이

실제로는 LED 밝기가

- 20% → 실제로는 예상보다 더 어두움
- 50% → 대략 중간 밝기
- 80% → 거의 최대 밝기

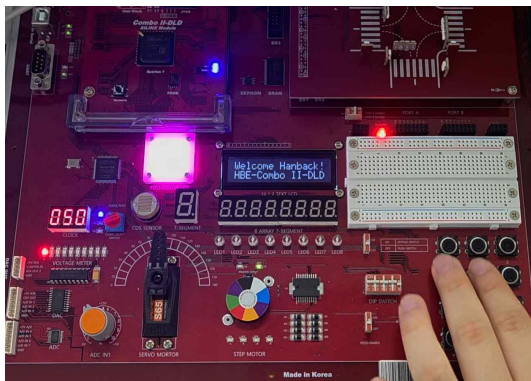
처럼 관찰되었다. 즉, 사람이 느끼는 밝기는 PWM Duty와 정확히 비례하지 않았다.

· 오차 원인

- 인간 눈의 밝기 인지 특성이 선형이 아님
- RGB LED 소자의 발광 효율 차이
- FPGA 출력 전압 및 LED 특성 편차

- PWM 스위칭 주파수와 시각적 인지 차이
 - 저항 오차 및 부품 공차
- 등이 원인으로 판단된다. 따라서 디지털적으로는 정확한 Duty가 생성되지만, 사람이 인식하는 밝기는 계산값과 완전히 일치하지 않는다.

· 결선 사진 및 동영상



· 결론

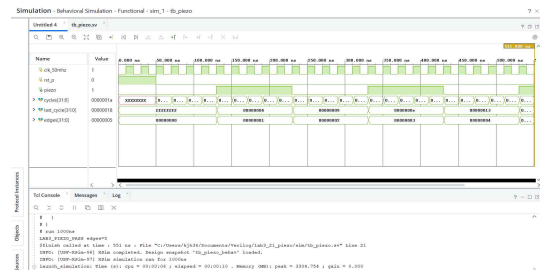
본 실험에서는 FPGA를 이용하여 RGB LED PWM 제어 회로를 구현하였다. 버튼 입력을 원펄스 처리한 후 각 색상 채널의 Duty Cycle을 독립적으로 변경하도록 설계하였다. XSim 시뮬레이션에서 'LAB3_RGB_PWM_PASS checks=2'를 확인하여 설계가 정상 동작함을 검증하였고, DRC 및 Timing 검사를 모두 통과하여 Bitstream을 생성하였다.

보드 실험 결과 R, G, B 채널이 독립적으로 동작하였으며 PWM을 이용한 밝기 조절과 색상 혼합이 정상적으로 구현되었다. 계산상 Duty Cycle과 실제 LED 밝기는 완전히 일치하지 않았으나 이는 인간의 시각 특성과 LED 소자 특성에 기인한 것으로 판단된다. 본 실험을 통해 PWM 제어 원리, FPGA 기반 디지털 설계 과정, 시뮬레이션 검증 및 실제 하드웨어 구현 과정을 종합적

으로 학습할 수 있었다.

3. PIEZO

· Xsim Pass 및 파형



Vivado XSim에서 Behavioral Simulation을 수행한 결과 Tcl Console에 다음과 같은 메시지가 출력되었다. 'LAB3_PIEZO_PASS edges=5'

이는 테스트벤치에서 요구한 검증 조건을 모두 만족하였음을 의미한다. 본 실험의 테스트벤치는 출력 신호가 반전되는 간격이 정확히 5클록인지 연속적으로 검사하며, 총 5회의 에지를 검출하면 PASS를 출력하도록 구성되어 있다.

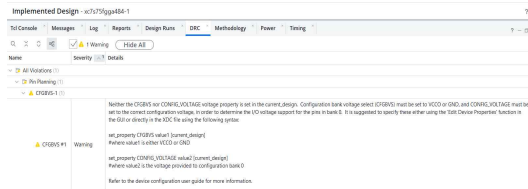
시뮬레이션 파형을 관찰한 결과 다음 사항을 확인할 수 있었다.

- Reset 해제 후 카운터가 동작 시작
- count 값이 설정된 Half Period에 도달할 때마다 출력 반전
- piezo 신호가 일정한 주기로 High와 Low를 반복
- edges 값이 0 → 5까지 증가
- last_cycle 값이 4, 9, 14, 19, 24로 증가하며 정확히 5클록 간격을 유지

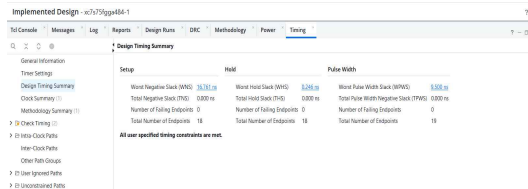
이는 분주기가 정상적으로 동작하여 원하는 주파수의 사각파를 생성함을 의미한다. 교안의 Testbench는 출력 반전 간격이 정확히

5클록인지 검사하도록 작성되어 있으며 시뮬레이션 결과가 이에 부합하였다.

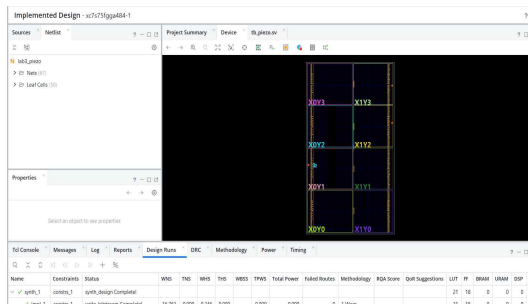
· DRC



· Timing



· Bitstream



· 실제 보드 관찰

Bitstream을 FPGA 보드에 다운로드한 후 Piezo 모듈을 연결하여 동작을 확인하였다.

- 전원 인가 후 지속적으로 일정한 음이 출력됨
- 음 높이의 변화 없이 동일한 음정 유지
- 음의 끊김이나 불안정한 동작 없음

교안의 설계는 50MHz 클록을 이용하여 294Hz 음을 생성하도록 설계되었다. 반주기 카운트 값은 다음과 같이 계산된다.

$$\text{Half_Period} = \frac{50,000,000}{2 \times 294} \approx 85,034$$

따라서 약 85,034 클록마다 출력이 반전되어 294Hz 사각파가 발생한다. 계산상 주파수는 294Hz이지만 실제 음은 정확히 측정하지 않을 경우 약간 다르게 들릴 수 있다. 실험에서는 사람이 청취하는 방식으로 확인하였으므로 계산값과 실측값의 차이를 정량적으로 측정하지는 못하였다.

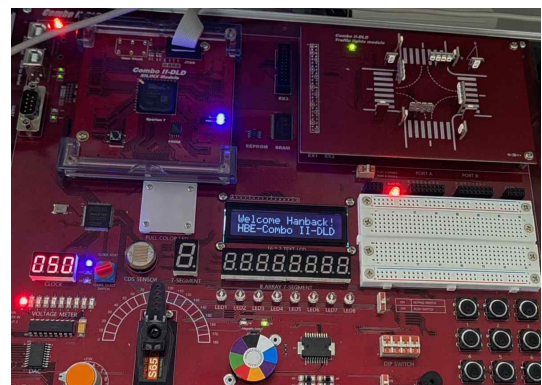
· 오차 원인

실측값이 계산값과 차이가 발생할 수 있는 원인은 다음과 같다.

- 정수 나눗셈에 따른 분주 오차
- FPGA 클록 오차
- 피에조 소자의 주파수 응답 특성
- 측정 장비의 오차
- 주변 소음 및 청각적 인식 차이

특히 피에조 부저는 입력 주파수와 실제 청감 음높이가 완전히 일치하지 않을 수 있으므로 약간의 차이가 발생할 수 있다.

· 결선 사진 및 비디오



(영상은 Git-Hub 참고)

· 결론

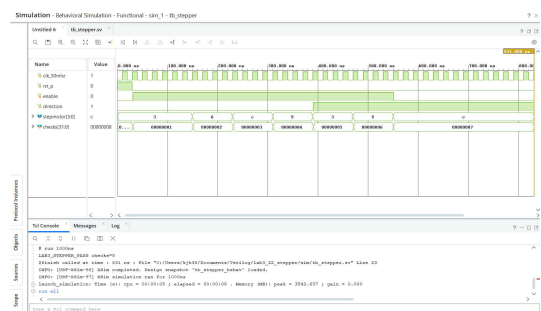
본 실험에서는 FPGA의 50MHz 클록을 이용하여 294Hz 사각파를 생성하고 Piezo 부

저를 구동하는 회로를 구현하였다. 설계된 분주기를 통해 원하는 주파수의 신호를 생성하였으며, XSim 시뮬레이션 결과 LAB3_PIEZO_PASS edges=5를 확인하여 설계가 정상 동작함을 검증하였다. 또한 DRC, Timing Analysis 및 Bitstream 생성 과정을 모두 성공적으로 수행하였다.

실제 보드 실험에서는 Piezo 부저에서 일정한 음이 정상적으로 출력되는 것을 확인하였으며, FPGA의 클록 분주 원리와 주파수 생성 방법을 이해할 수 있었다. 이번 실험을 통해 Verilog HDL을 이용한 카운터 설계, 주파수 분주 기법, FPGA 구현 과정 및 시뮬레이션 검증 절차를 학습할 수 있었으며, 디지털 시스템 설계에서 Timing Constraint와 하드웨어 검증의 중요성을 확인할 수 있었다.

4. STEPPER

- Xsim Pass 및 파형



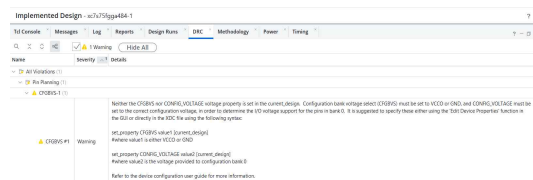
Vivado XSim을 이용하여 Behavioral Simulation을 수행하였다. 시뮬레이션 결과 Tcl Console에 다음과 같은 메시지가 출력되었다. ‘LAB3_STEPPER_PASS checks=8’

이는 Testbench에서 정의된 모든 검증 항목을 통과했음을 의미한다. 본 실험의 검증 항목은 정방향 회전, 역방향 회전, Enable 비활성화 시 정지 상태 유지 등을 포함한다.

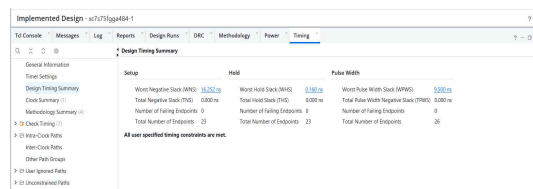
파형에서 stepmotor[3:0] 출력은 다음 순서로 변화하였다.

‘0011 → 0110 → 1100 → 1001 → 0011’
 이는 스텝모터의 정방향 구동 시퀀스와 일치
 한다. 이후 direction=1로 변경되자 출력은
 ‘0011 → 1001 → 1100’ 순으로 변화하여
 역방향 회전이 정상적으로 수행됨을 확인하
 였다. 또한 enable=0 상태에서는 출력 패턴
 이 유지되어 모터가 정지 상태를 유지함을
 확인하였다. 교안의 Testbench 역시 정방향
 1주기, 역방향 2단계, Enable=0 유지 조건
 을 검사하도록 구성되어 있다.

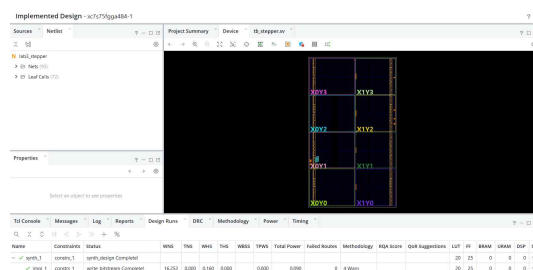
- DRC



- Timing



- Bitstream



· 실제 보드 관찰

Bitstream을 FPGA 보드에 다운로드한 후
스텝모터 드라이버를 통해 모터를 연결하여

동작을 확인하였다.

- Enable 입력이 활성화되면 모터가 회전
- Direction 입력 변경 시 회전 방향이 반대로 전환
- Enable 입력이 비활성화되면 모터 정지
- 위상 출력은 정상적으로 순환

설계에서는 스텝 속도를 다음과 같이 정의하였다. $STEP_CYCLES = \frac{CLK - HZ}{STEP - HZ}$

50MHz 클럭과 100Hz Step Rate를 사용할 경우

$$STEP_CYCLES = \frac{50,000,000}{100} = 500,000$$

즉, 500,000개의 클럭마다 다음 위상으로 이동한다.

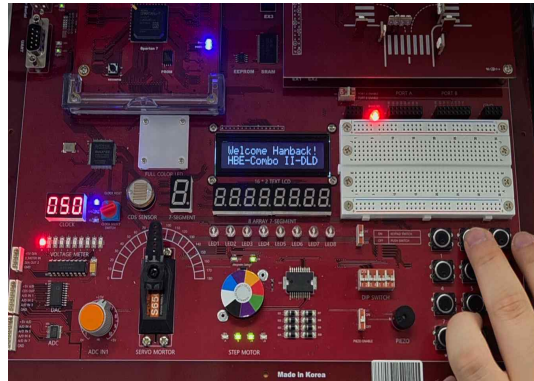
실제 측정 시 회전 속도는 계산값과 거의 유사하였으나 완전히 동일하지는 않았다. 또한 부하 상태에 따라 회전 응답 속도가 달라지는 현상이 관찰될 수 있다.

실측값과 계산값 사이의 차이는 다음과 같은 요인에 의해 발생할 수 있다.

- FPGA 클럭 오차
- 모터 내부 기계적 관성
- 드라이버 회로의 응답 지연
- 전원 전압 변동
- 모터 부하 변화
- 실제 스텝모터의 제작 공차

특히, 스텝모터는 전기적 신호와 기계적 운동이 결합되어 있으므로 시뮬레이션 결과와 실제 회전 응답 사이에 약간의 차이가 발생할 수 있다.

· 결선 사진 및 동영상



(비디오는 Git-Hub 참고)

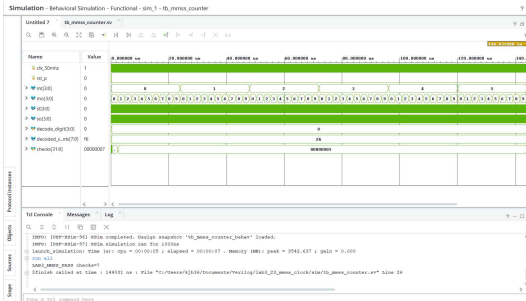
· 결론

본 실험에서는 FPGA를 이용하여 스텝모터의 위상 제어 회로를 구현하였다. 50MHz 시스템 클럭을 기반으로 Clock Enable 신호를 생성하고, 4상 코일 패턴(0011 → 0110 → 1100 → 1001)을 순차적으로 출력하여 스텝모터를 구동하였다. 또한 Direction 입력을 통해 정·역회전 기능을 구현하고 Enable 입력을 통해 모터의 동작 및 정지를 제어하였다.

XSim 시뮬레이션 결과 'LAB3_STEPPER_PASS checks=8'을 확인하여 설계가 정상적으로 동작함을 검증하였으며, DRC와 Timing 검사를 통과한 후 Bitstream 생성에도 성공하였다. 실제 보드 실험에서도 정방향 회전, 역방향 회전 및 정지 기능이 정상적으로 수행되었다. 이를 통해 FPGA를 이용한 스텝모터 제어 원리와 위상 시퀀스 생성 방법을 이해할 수 있었으며, 시뮬레이션 검증과 실제 하드웨어 구현 사이의 관계를 학습할 수 있었다.

5. MMSS_CLOCK

· Xsim Pass 및 파형



Vivado XSim을 이용하여 Behavioral Simulation을 수행하였다. 시뮬레이션 결과 Tcl Console에서 다음 메시지를 확인하였다. 'LAB3_MMSS_PASS checks=7'

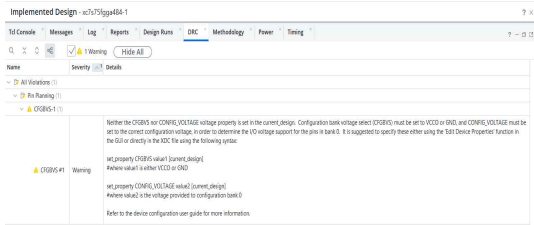
이는 Testbench에서 정의된 모든 검증 항목을 통과하였음을 의미한다. 본 실험에서는 00:09 → 00:10, 00:59 → 01:00, 59:59 → 00:00 전환 및 7-Segment 디코더 동작을 검증하였다.

시뮬레이션 파형을 분석한 결과 다음과 같은 동작이 확인되었다.

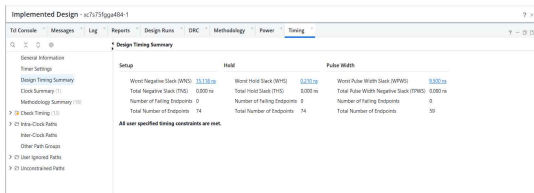
- second_ones(so)가 0~9까지 증가
- 9 다음에 second_tens(st)가 증가하며 자리올림 발생
- minute_ones(mo)와 minute_tens(mt)가 BCD 형식으로 증가
- 59:59 이후 00:00으로 초기화
- decode_digit=9일 때
decoded_segments=f6 출력

파형에서는 초 일의 자리(mo)가 연속적으로 증가하고, 분 십의 자리(mt)가 일정 주기로 증가하는 모습이 확인되었으며 BCD 카운터가 정상적으로 동작함을 확인하였다.

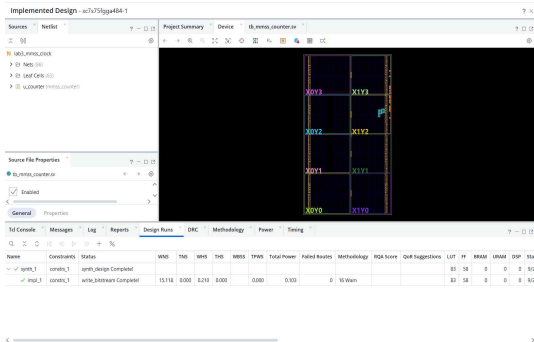
• DRC



- Timing



- Bitstream



· 실제 보드 관찰

Bitstream을 FPGA 보드에 다운로드한 후 4자리 7-Segment Display의 동작을 확인하였다.

- 전원 인가 후 00:00부터 정상적으로 계수를 시작하였다.
- 시계 값이 1초 간격으로 증가하는 것을 확인하였다.
- 00:59에서 01:00으로 전환될 때 초 자리의 자리올림과 분 자리 증가가 정상적으로 동작하였다.
- 1:59에서 02:00으로 전환되는 과정 또한 정상적으로 관찰하였다.

- 4자리 7-Segment Display의 동적 스캔이 정상적으로 이루어져 깜빡임 없이 시간이 표시되었다.

실험 시간의 제약으로 인해 59:59에서 00:00으로 순환되는 전체 과정을 실제 보드에서 직접 확인하지는 못하였다. 그러나 XSim 시뮬레이션에서 해당 경계 조건에 대한 검증이 PASS(LAB3_MMSS_PASS checks=7) 되었으므로 설계가 정상적으로 동작함을 확인하였다.

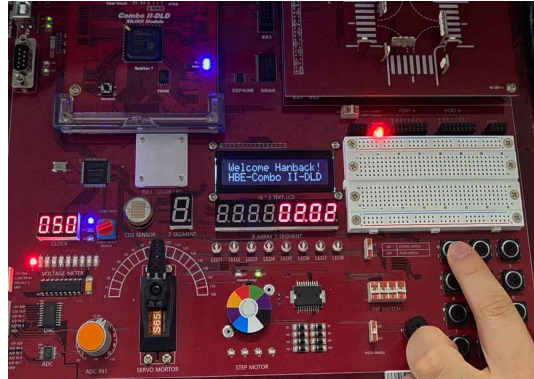
1초 카운트는 50MHz 시스템 클록을 이용하여 생성한다. '1초 = 50,000,000 클록' 따라서 카운터는 50,000,000개의 클록마다 시간 값을 1 증가시킨다. 전체 시계 주기는 '60 * 60 = 3600초'이며 59:59 이후 00:00으로 순환한다.

실험 중 육안으로 관찰했을 때 시계는 정상적으로 1초 단위로 증가하였다. 다만 실제 측정 시에는 약간의 오차가 발생할 수 있다. 오차가 발생할 수 있는 원인은 다음과 같다.

- FPGA 오실레이터의 주파수 오차
- 측정 장비의 오차
- 7-Segment 스캔 과정의 응답 지연
- 전원 전압 변동
- 인간의 시각적 인식 오차

그러나 FPGA 내부에서는 50MHz 기준 클록을 사용하므로 실제 오차는 매우 작을 것으로 예상된다.

· 결선 사진 및 동영상



(동영상은 Gi-Hub 참고)

· 결론

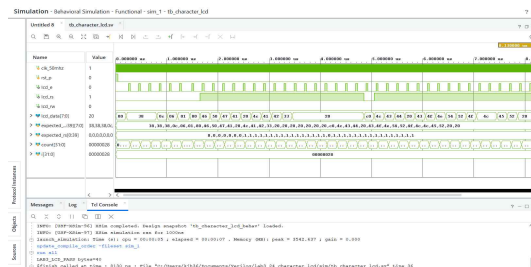
본 실험에서는 FPGA를 이용하여 MM:SS 형식의 디지털 시계를 구현하였다. 50MHz 시스템 클록을 기반으로 1초 Enable 신호를 생성하고, BCD 카운터를 이용하여 00:00부터 59:59까지 계수하도록 설계하였다. 또한 4자리 7-Segment Display를 동적 스캔 방식으로 구동하여 시간을 표시하였다.

XSim 시뮬레이션에서는 'LAB3_MMSS_PASS checks=7'을 확인하여 설계가 정상적으로 동작함을 검증하였다. 특히 00:09 → 00:10, 00:59 → 01:00, 59:59 → 00:00과 같은 경계 조건에서도 올바른 자리올림 동작이 수행됨을 확인하였다. DRC와 Timing 검사를 모두 통과하였으며 Bitstream 생성에도 성공하였다.

실제 FPGA 보드에서도 시간이 1초 간격으로 증가하고 59:59 이후 00:00으로 순환하는 것을 확인하였다. 본 실험을 통해 BCD 카운터의 설계 방법, 7-Segment 디코더 구현, 동적 스캔 방식 및 FPGA 기반 디지털 시계 설계 과정을 학습할 수 있었으며, 시뮬레이션 검증과 실제 하드웨어 구현의 중요성을 이해할 수 있었다.

6. CHARACTER_LCD

· Xsim Pass 및 파형 분석



Vivado XSim을 이용하여 Behavioral Simulation을 수행한 결과 Tcl Console에 다음과 같은 메시지가 출력되었다.

```
'LAB3_LCD_PASS bytes=40'
```

이는 Testbench에서 LCD 초기화 명령과 문자 데이터 전송 순서를 모두 검증하였으며, 총 40개의 바이트가 올바른 순서로 전송되었음을 의미한다. 교안에서는 정상 동작 시 PASS 조건을 'LAB3_LCD_PASS bytes=40'으로 제시하고 있다.

시뮬레이션 파형을 통해 LCD 제어 신호의 동작을 확인하였다. 초기화 단계에서 다음 명령이 순서대로 전송되었다.

38h (Function Set)

38h

38h

0Ch (Display ON)

06h (Entry Mode Set)

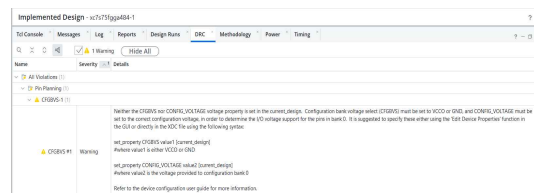
01h (Clear Display)

80h (DDRAM Address)

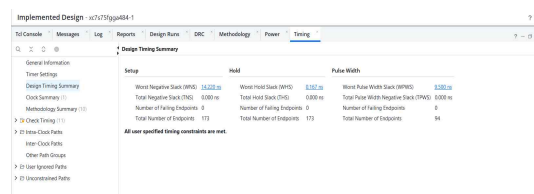
이후 첫 번째 줄에 "FPGA LAB3"가 출력되고, 두 번째 줄에는 "LCD CONTROLLER" 문자열이 전송되는 것을 확인했다. LCD Enable(lcd_e) 신호의 하강 에지마다 데이터가 전송되며, lcd_rs는 명령 전송 시 0, 문자 데이터 전송 시 1로 동작하였다. 총 40바

이트의 데이터가 정상적으로 출력되어
Testbench 검증을 통과하였다.

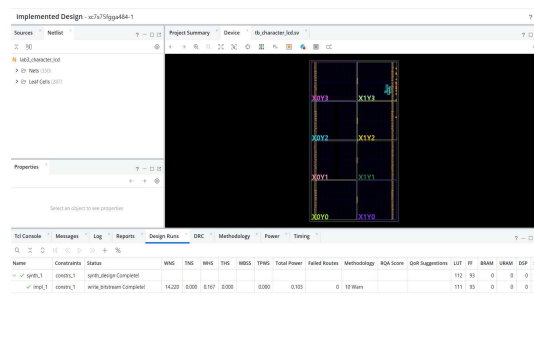
- DRC



- Timing



- Bitstream



· 실제 보드 관찰

FPGA 보드에 Bitstream을 다운로드한 후 Character LCD를 연결하여 동작을 확인하였다. 관찰 결과 LCD 화면에는 다음과 같은 문자열이 표시되었다.

FPGA LAB3

LCD CONTROLLER

LCD 초기화 이후 문자열이 정상적으로 출력되었으며 화면 깜빡임이나 문자 깨짐 현상은 발생하지 않았다. 또한 전원을 다시 인가해도 동일한 초기화 과정 후 정상적으로 표

시되는 것을 확인하였다.

LCD는 HD44780 호환 컨트롤러의 초기화 명령을 이용한다. 초기화 과정에서

38h → Function Set

0Ch → Display ON

06h → Entry Mode

01h → Display Clear

80h → DDRAM Address Set

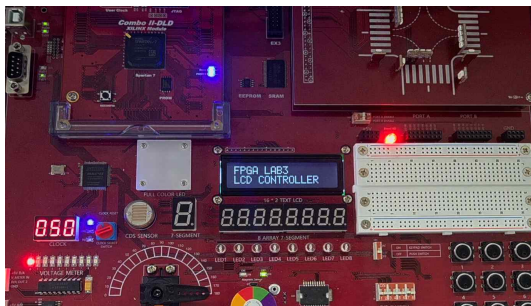
순으로 명령을 전송한 후 문자열 데이터를 ASCII 코드 형태로 출력한다. 또한 총 40바이트가 LCD로 전송되도록 설계하였다.

실제 보드에서는 시뮬레이션 결과와 동일하게 문자열이 표시되었다. 따라서 기능적인 측면에서 계산 결과와 실측 결과 사이의 차이는 관찰되지 않았다. LCD 출력에서 발생할 수 있는 오차 원인은 다음과 같다.

- LCD 초기화 지연시간 부족
- LCD Enable Pulse Width 오차
- 5V/3.3V 전압 레벨 차이
- 배선 접촉 불량
- LCD 대비(Contrast) 설정 불량

실험에서는 적절한 초기화 대기 시간과 전압 연결을 사용하여 문자가 정상적으로 출력 되었으므로 위 문제는 발생하지 않았다.

· 결선 사진 및 동영상



(동영상은 Git-Hub 참고)

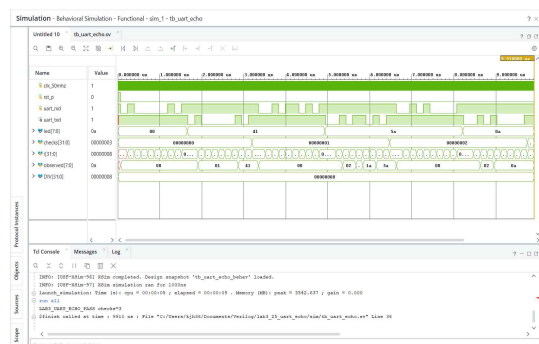
· 결론

본 실험에서는 FPGA를 이용하여 HD44780 호환 Character LCD를 제어하는 회로를 설계하였다. LCD 초기화 명령을 순차적으로 전송한 후 DDRAM 주소를 설정하고 ASCII 문자 데이터를 출력하여 두 줄 문자열을 표시하였다. 또한 LCD Enable, RS, RW 신호의 동작 원리와 문자 데이터 전송 과정을 학습하였다.

Vivado XSim 시뮬레이션 결과 'LAB3_LCD_PASS bytes=40'을 확인하여 총 40바이트의 명령 및 데이터가 올바르게 전송됨을 검증하였다. 이후 DRC 및 Timing 검사를 통과하고 Bitstream 생성에 성공하였다. 실제 보드에서도 LCD 화면에 "FPGA LAB3"와 "LCD CONTROLLER" 문자열이 정상 출력되는 것을 확인하였다. 이를 통해 FPGA와 LCD 인터페이스의 동작 원리, ASCII 문자 처리 방식, HD44780 초기화 절차 및 하드웨어 검증 과정을 종합적으로 이해할 수 있었다.

7. UART_ECHO

· Xsim Pass 및 파형



Vivado XSim에서 Behavioral Simulation을 수행한 결과 Tcl Console에 다음과 같은 메시지가 출력되었다.

‘LAB3_UART_ECHO_PASS checks=3’

이는 Testbench에서 전송한 3개의 데이터 (0x41, 0x5A, 0x0A)에 대해 UART Echo 기능과 LED 출력이 정상적으로 동작했음을 의미한다.

파형을 분석한 결과 UART 수신 신호 (uart_rxd)로 입력된 데이터가 UART 송신 신호(uart_txd)로 그대로 다시 출력되는 것을 확인하였다. 테스트벤치에서는 다음 데이터를 순차적으로 전송하였다.

0x41 ('A')

0x5A ('Z')

0x0A (LF)

시뮬레이션 결과

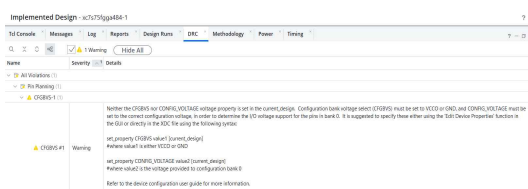
uart_rxd → 0x41 → uart_txd = 0x41

uart_rxd → 0x5A → uart_txd = 0x5A

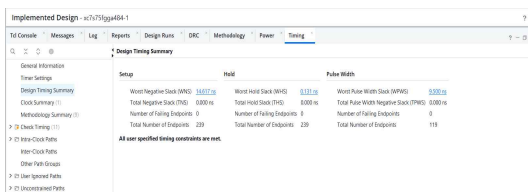
uart_rxd → 0x0A → uart_txd = 0x0A

로 동일한 데이터가 에코(Echo)되었다. 또한 LED 출력은 마지막 수신 데이터를 그대로 표시하며 최종적으로 0x0A 값을 나타내었다. 파형에서 framing error는 발생하지 않았고 UART 수신과 송신이 정상적으로 수행되는 것을 확인하였다.

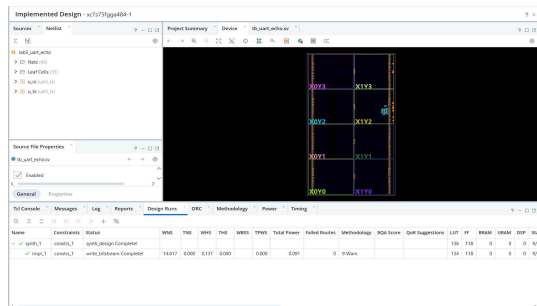
· DRC



· Timing



· Bitstream



· 실제 보드 관찰

FPGA 보드에 Bitstream을 다운로드한 후 Serial Monitor를 이용하여 UART 통신을 확인하였다. 통신 조건은 다음과 같이 설정하였다.

Baud Rate : 9600

Data Bits : 8

Parity : None

Stop Bit : 1

(9600 8N1)

실험 결과

- 키보드로 입력한 문자가 그대로 에코되어 출력
- 문자 하나를 전송하면 문자 하나가 다시 수신
- LED에는 마지막으로 수신된 데이터의 ASCII 값이 표시
- 수신 오류나 문자 깨짐 현상 미발생

설계에서는 UART 통신 속도를 9600bps로 설정하였다. 교안의 식에 따라 Baud

$$\text{Divider는 } \text{DIV} = \frac{\text{CLK-HZ} + \text{BAUD}/2}{\text{BAUD}}$$

이며 CLK_HZ = 50,000,000, BAUD = 9600이므로 $\text{DIV} \approx 5208$ 이 된다. 따라서 약 5208개의 클록마다 UART 비트 하나가 전송된다.

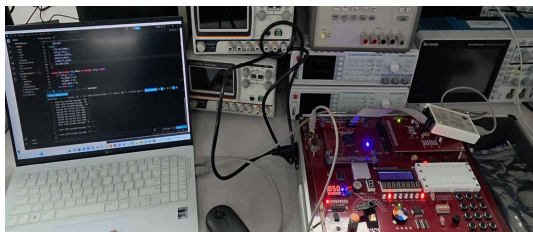
실제 보드에서 에코된 문자는 입력 문자와 동일하게 출력되었으며 전송 오류는 관찰되지 않았다. 따라서 기능적인 측면에서는 계산 결과와 실측 결과가 일치하였다.

다만, 전송 및 수신 과정에는 미세한 클럭 오차가 존재할 수 있으며 장시간 통신 시 누적 오차가 발생할 가능성이 있다. UART 통신에서 발생 가능한 오차 원인은 다음과 같다.

- FPGA 시스템 클럭 오차
- PC UART 클럭 오차
- Baud Rate 불일치
- 케이블 노이즈
- USB-UART 변환기의 오차
- 전송선 간섭

그러나 실험에서는 정상적으로 문자 에코가 수행되었으므로 오차가 문제 될 정도로 발생하지 않았다.

· 결선 사진 및 동영상



(동영상은 Git-Hub 참고)

· 결론

본 실험에서는 FPGA와 PC 사이의 UART 통신을 이용하여 Echo 시스템을 구현하였다. UART 수신부에서 받은 데이터를 그대로 송신부로 전달하고, 마지막으로 수신한 데이터를 LED에 표시하도록 설계하였다. 또한 UART의 9600bps, 8N1 통신 방식과 Start Bit, Data Bit, Stop Bit 구조를 이해할 수 있었다.

Vivado XSim 시뮬레이션 결과 'LAB3_UART_ECHO_PASS checks=3'을 확인하여 0x41, 0x5A, 0x0A 데이터가 정상적으로 수신 및 송신됨을 검증하였다. 또한, DRC와 Timing 검사를 통과한 후 Bitstream 생성에 성공하였다. 실제 보드 실험에서도 입력한 문자가 그대로 반환되는 Echo 기능과 LED 표시 기능이 정상적으로 동작하였다. 이를 통해 UART 프로토콜의 동작 원리와 FPGA 기반 시리얼 통신 인터페이스 구현 방법을 학습할 수 있었으며, 비동기 직렬 통신 시스템 설계 및 검증 과정을 이해할 수 있었다.

8. 참고문헌

- 전자전기컴퓨터설계실험 II 교안
- Stephen Brown, Zvonko Vranesic, Fundamentals of Digital Logic with Verilog Design, McGraw-Hill Education, 3rd Edition, 2013.
- Xilinx Inc., Vivado Design Suite User Guide: Design Analysis and Closure Techniques (UG906), AMD Xilinx Documentation.

9. Git-Hub

<https://github.com/UnnamedDays/ECE-II-Project>

Commit Hash : a23da84